Lab 13.1 - Template & Deploy with cloud-init

KVM Virtualisation on Oracle Linux - Day 2, Module 13 (VM Templating)

Overview

Every VM built so far in this course has started from the same generic Oracle base image, every time. Real environments often work differently: customise a VM once - install packages, apply configuration - then turn that customised VM itself into a reusable template, and deploy as many fresh instances from it as needed, each with its own identity applied automatically at boot.

The tricky part is identity. A naive copy of a customised VM carries the original's SSH host keys, machine-id, and cloud-init's own memory of having already run - clone it carelessly and you get two machines that look identical to anything checking their fingerprint, which is a real problem the moment both are on the same network. This lab uses virt-sysprep to strip that identity out of a template, and cloud-init to give each deployment a fresh one back - the same NoCloud seed pattern used since Lab 3.1, now applied to a golden image workflow instead of a single VM build.

Learning Objective

By the end of this lab, you will be able to:

- Explain why a template must have its machine-specific identity removed before being reused
- Generalise a VM disk into a reusable template with virt-sysprep
- Write a cloud-config declaring a new hostname and SSH key for a fresh deployment
- Launch a new VM from a template with that configuration attached
- Confirm hostname, login access, and identity all applied automatically, with no manual login required to configure them

Step by Step Guide

Prerequisite: Connect to Your Lab VM

1. Connect to your lab host over SSH:

ssh opc@<your-lab-instance-ip>

2. Confirm ol-lab-01 exists:

sudo virsh list --all

Step 1: Shut Down a Copy of ol-lab-01 to Use as the Master

The template is built from a copy of ol-lab-01's disk, not ol-lab-01 itself - this leaves the original VM untouched and available for every other lab, while the copy becomes a disposable, reusable master image.

3. Shut ol-lab-01 down so its disk can be safely copied:

```
sudo virsh shutdown ol-lab-01
watch -n 2 sudo virsh list --all
# Ctrl-C once state shows "shut off"
```

4. Copy its disk to a new master template file:

```
sudo cp /var/lib/libvirt/images/ol-lab-01.qcow2 \
  /var/lib/libvirt/images/ol-lab-01-master.qcow2
```

5. Start ol-lab-01 back up, since the copy is all this lab needs from here on:

```
sudo virsh start ol-lab-01
```

Step 2: Generalise the Master into a Template with virt-sysprep

virt-sysprep - from the same libguestfs family as virt-customize and virt-cat used earlier - strips exactly the machine-specific data that would otherwise make every deployment from this template indistinguishable from every other.

6. Confirm virt-sysprep is available - it ships in the same package as virt-customize:

```
which virt-sysprep || sudo dnf install -y libguestfs-tools-c
```

7. Run it against the master copy, using its default operation set:

```
sudo virt-sysprep -a /var/lib/libvirt/images/ol-lab-01-master.qcow2
```

Expected result: virt-sysprep reports each operation it performed as it runs, then finishes cleanly. Among the default operations: ssh-hostkeys (removed, so each deployment generates its own fresh host keys on first boot), machine-id (cleared), cloud-init (cloud-init's own record of having already run is wiped, so it genuinely treats next boot as first boot), logfiles, bash-history, and udev-persistent-net (removed).

Note: If this fails with an appliance error, the same LIBGUESTFS_BACKEND=direct workaround from earlier virt-customize troubleshooting applies here too - virt-sysprep uses the identical libguestfs appliance mechanism underneath.

8. ol-lab-01-master.qcow2 is now the reusable template - confirm its machine-id is genuinely gone, not just reset to a placeholder:

```
sudo virt-cat -a /var/lib/libvirt/images/ol-lab-01-master.qcow2 \
  /etc/machine-id
```

Expected result: Either an empty file or a file that doesn't exist yet - the guest will generate a genuinely new one itself on first boot, which is what makes each deployment unique.

Step 3: Launch a New VM from the Template

Deploy a fresh VM from the template, with its own seed ISO declaring a new hostname and the same SSH key used throughout this course.

9. Copy the template to give this specific deployment its own independent disk - the template file itself is never used directly, so it stays reusable for the next deployment too:

```
sudo cp /var/lib/libvirt/images/ol-lab-01-master.qcow2 \
  /var/lib/libvirt/images/app-vm-01.qcow2
```

10. Build a seed ISO declaring the new VM's identity:

```
mkdir -p ~/app-vm-01-seed && cd ~/app-vm-01-seed
cat > meta-data << EOF
instance-id: app-vm-01-v1
local-hostname: app-vm-01
EOF
cat > user-data << EOF
#cloud-config
ssh_authorized_keys:
  - $(cat ~/.ssh/id_ed25519.pub)
runcmd:
  - nmcli connection reload
  - nmcli connection up eth0
EOF
```

11. Build the ISO, preferring xorrisofs but falling back to xorriso if it isn't present:

```
if command -v xorrisofs >/dev/null 2>&1; then
  sudo xorrisofs -output /var/lib/libvirt/images/app-vm-01-seed.iso \
```

```
  -volid cidata -joliet -rock user-data meta-data
else
  sudo xorriso -as mkisofs -output /var/lib/libvirt/images/app-vm-01-seed.iso \
    -volid cidata -joliet -rock user-data meta-data
fi
```

Note: This seed has no network-config file, unlike earlier labs - with none provided, cloud-init falls back to a plain DHCP request on whatever interface it finds, which is exactly what's wanted here since this VM is going on the same default NAT network ol-lab-01 originally used.

12. Create the new VM, importing the deployed disk with the seed ISO attached:

```
sudo virt-install \
  --name app-vm-01 \
  --memory 1024 --vcpus 1 \
  --disk path=/var/lib/libvirt/images/app-vm-01.qcow2,format=qcow2 \
  --disk path=/var/lib/libvirt/images/app-vm-01-seed.iso,device=cdrom \
  --import --os-variant ol9.0 \
  --network network=default \
  --graphics vnc,listen=0.0.0.0 --noautoconsole
```

Expected result: Domain creation completed. sudo virsh list --all shows app-vm-01 running.

Step 4: Confirm Hostname, User, and Key All Took - No Manual Login

Verify everything applied automatically at boot, without ever logging in to configure anything by hand.

13. Find the new VM's address:

```
sudo virsh domifaddr app-vm-01
```

14. Confirm the hostname, login user, and sudo access all took, in one command:

```
ssh -i ~/.ssh/id_ed25519 cloud-user@<app-vm-01's address> \
  'hostname; whoami; sudo whoami'
```

Expected result: Three lines back: app-vm-01, cloud-user, and root - the hostname from meta-data, the account the SSH key was delivered to, and confirmation that account has working sudo access. No password was typed, no console login was needed.

15. Confirm this VM's identity is genuinely its own, not a copy of ol-lab-01's:

```
ssh -i ~/.ssh/id_ed25519 cloud-user@<app-vm-01's address> \
```

'cat /etc/machine-id'

ssh -i ~/.ssh/id_ed25519 cloud-user@<ol-lab-01's address> \

'cat /etc/machine-id'

Expected result: Two different machine-id values - proof virt-sysprep's cleanup and cloud-init's fresh-boot handling genuinely produced a distinct machine, not a clone wearing a new hostname on top of an identical underlying identity.

Outcomes

By the end of this lab, you have a reusable template (ol-lab-01-master.qcow2) with all machine-specific identity stripped out, and a freshly deployed VM (app-vm-01) built from it with its hostname, SSH access, and machine-id all applied automatically through cloud-init - no manual configuration step required after virt-install returned. The template itself is untouched and ready for the next deployment, which is the entire point of building it this way rather than customising each VM from scratch.